

SISTEMAS MICROPROCESADORES

MAQUETA DE LA BOLERA



EQUIPO 1

Lorena Rodríguez Carbia 18305

Gabriela Artaza Esteban 18759

Camino Luque Bayón 18732

Javier Sanz Parada 18329

Óscar Flórez Gil 17155

Pablo Lecoq Rael 15233

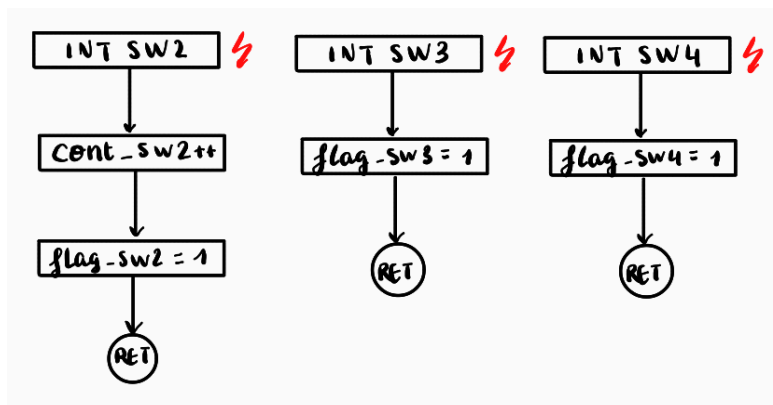
1. PLANTEAMIENTO DE LA SOLUCIÓN

Parte 1: Control de lanzador. Movimiento y lanzamiento de la bola y su luz.

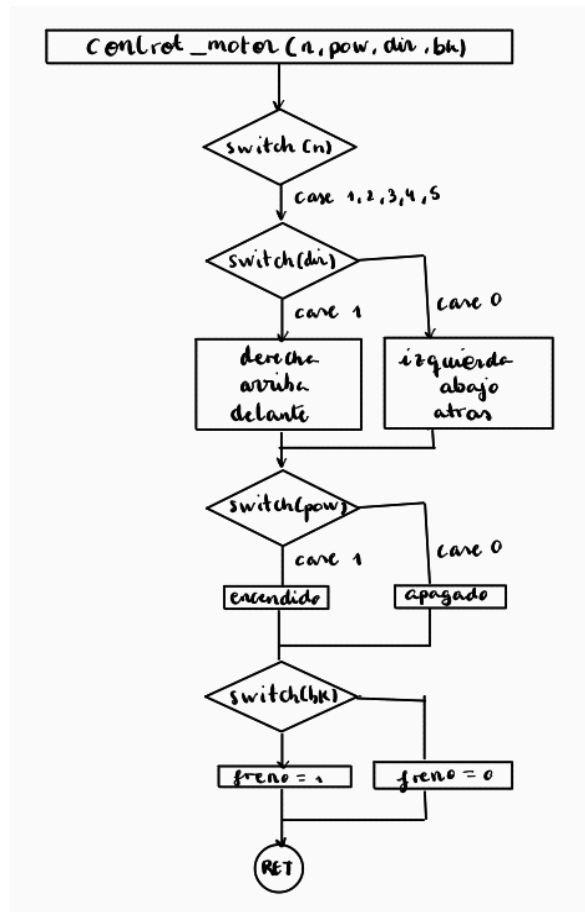
Integrantes: Óscar Flórez y Javier Sanz

Se ha decidido usar interrupciones para los sensores SW2, SW3, SW6 que controlan el movimiento de los motores M2, M3 Y M4. Ponen a 1 su flag cuando llega un flanco de bajada en el sensor. Este flag se usa para cambiar el sentido en algunas funciones o simplemente para el motor al llegar a su final de carrera.

Debido a los rebotes encontrados en los sensores hemos implementado **antirrebotes** en todos ellos para el correcto funcionamiento del programa, y se ha usado también una función para hacer **delays** de n milisegundos, como se vió en las prácticas de la asignatura.

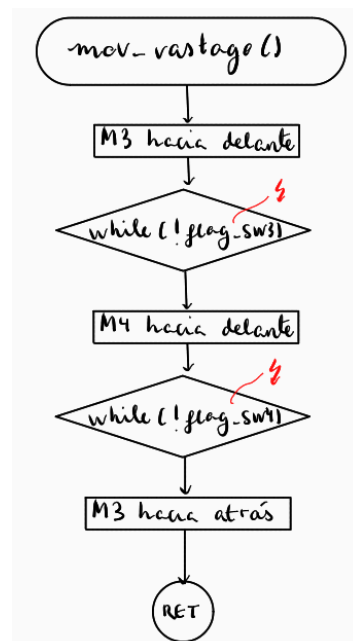
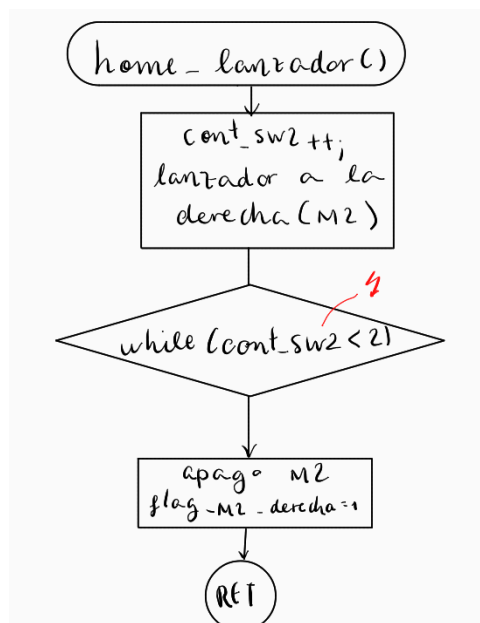


Para el control de los motores hemos realizado la función **control_motor()**. Como parámetros de entrada tiene el motor que queremos controlar, su enable correspondiente, así como la dirección y la señal brake que solo se usa en el motor 2 para frenar. La función consiste en un switch anidado con otros switches y varios casos para poder lograr el objetivo, de manera que se pueden controlar los 5 motores y sus direcciones con la misma función.

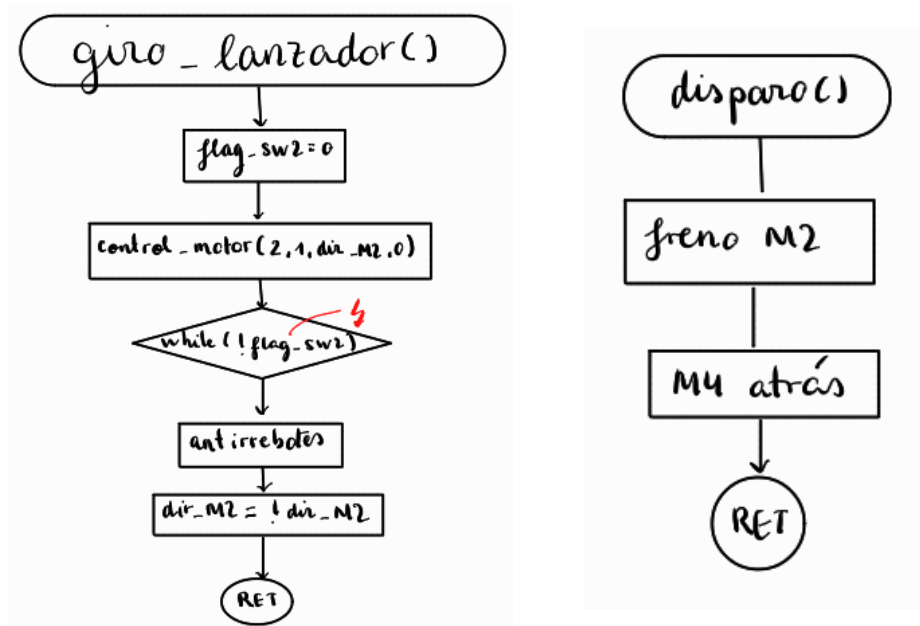


Para el movimiento del lanzador se han implementado las siguientes funciones:

- **home_lanzador():** lleva el motor M2 a la derecha del todo y lo pone en posición para que el motor M1 cargue una bola.
- **mov_vastago():** tensa la goma (M3) y pone la retención (M4) después de cargar la bola.

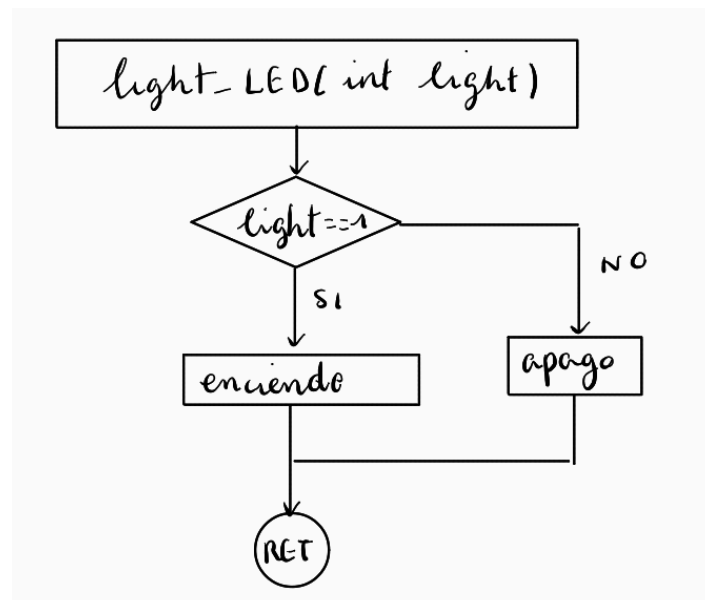


- **lanzador_izquierda()**: mueve el motor a la izquierda del todo, posición que hemos usado como inicialización para hacer el balanceo del motor. (Tiene el mismo esquema que home_lanzador pero en sentido contrario).
- **giro_lanzador()**: realiza el movimiento oscilante del lanzador en la posición de disparo esperando a que el jugador pulse el botón para disparar. Cambia de sentido mediante una interrupción en el sw2.
- **disparo()**: una vez pulsado el botón de disparo esta función se encarga de soltar la retención para soltar la bola.



Para el encendido y apagado del **LED** se usa la función:

- **light_LED()**:



Con intención de favorecer el proceso de integración hay una serie de banderas encargadas de indicar la posición de los elementos de la maqueta, estas banderas se pasan a integración mediante funciones que retornan el valor de dicha bandera.

Estas funciones son: **get_derecha_M2()**, **get_izquierda_M2()**, **get_vastago_atras()**.

Parte 2: Control del sistema de retorno de las bolas y del interruptor de disparo:

Integrantes: Lorena Rodríguez y Gabriela Artaza

En esta parte del código se controlan los motores 1 y 5 y el pulsador (SW6).

Setup_retorno():

Primero, se hace el setup de las entradas y salidas y después el setup de las interrupciones usadas:

Pulsador SW6:

Este botón se debe pulsar tanto para comenzar una partida, como para lanzar la bola.

El microinterruptor mecánico lo hemos conectado al pin PK0 del puerto K, el cual tiene una interrupción PCINT16.

SW1:

Está conectado al pin PD0 del puerto D, al cual le corresponde una interrupción de tipo INT0. Esta interrupción se ha configurado para que sea de flanco de bajada.

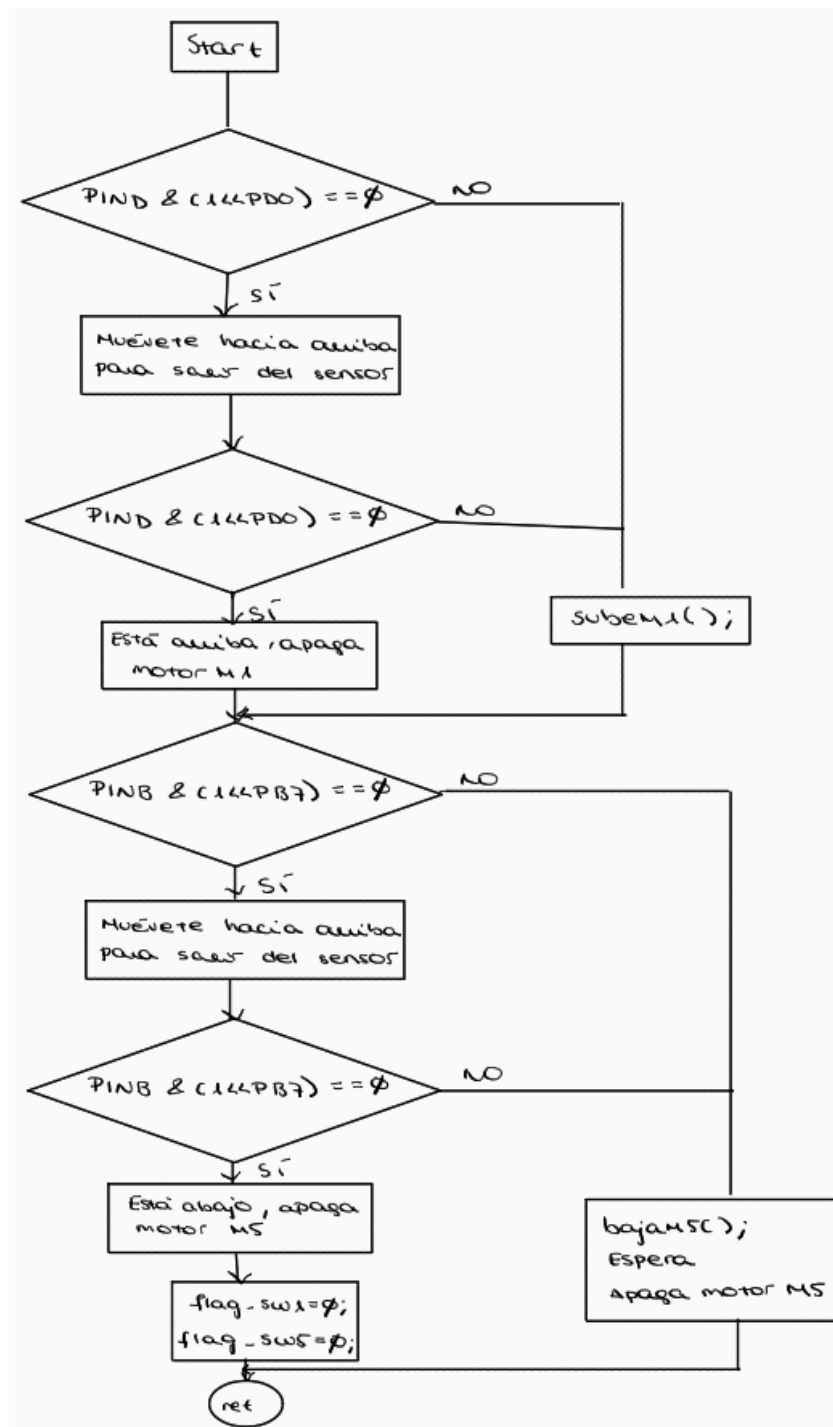
SW5:

Está conectado al pin PB7 del puerto B, por lo que tiene una interrupción de tipo PCINT7.

Setup_PWM():

Para ello, se ha elegido el modo FAST PWM de 8 bits, para poder hacer la comparación del OCR1A y OCR1B, con un preescalado de 1024. Así obtenemos un OCR de aproximadamente 255, para la frecuencia de 30Hz de los motores. Este OCR se ha definido previamente en el archivo *"macros.h"*.

Función start():



Esta función controla la inicialización de los motores 1 y 5. Al comenzar, el motor 1 debe moverse hacia arriba y el motor 5 debe desplazar el ascensor a abajo, quedando ambos motores en dichas posiciones.

Hemos vislumbrado la posibilidad de que cada uno de los motores se encuentre en tres posiciones distintas. Bien tocando el sensor de arriba, tocando el de abajo o no tocando ninguno, es decir, que se encuentre en la mitad del trayecto.

Para poder ver dónde se encuentran, se empieza con una comparación para ver si se está pulsando el pin PD0 (que es donde está el sensor SW1). Si es así, se desplaza un poco el motor 1 hacia arriba, se espera, se para el motor, y se vuelve a comparar. Si estando en esta posición el pin PD0 está pulsado, significa que ya estaba arriba,

por lo que se apaga el motor, para que no se sobrecargue.

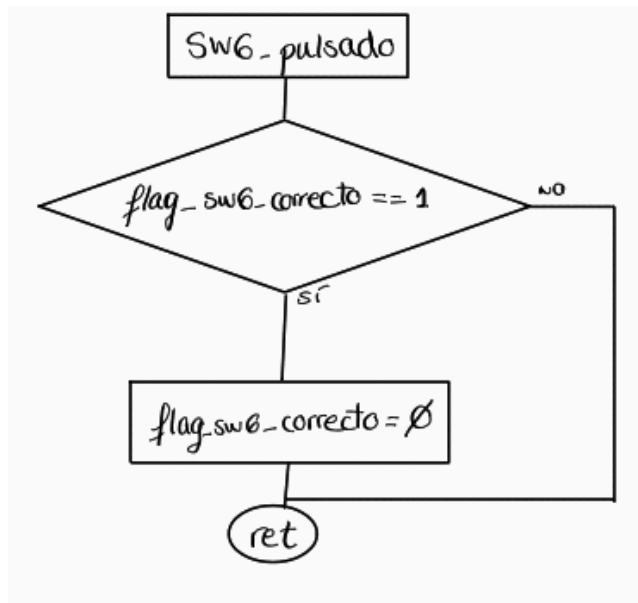
Si no, significa que antes estaba abajo y ahora está en algún lado entre arriba y abajo, por lo que se lleva el motor 1 a arriba del todo.

Si desde el principio no se está pulsando el PD0, se mueve hacia arriba.

La misma inicialización sirve para el motor 5, pero al contrario. Es decir, en vez de subirlo, se baja.

Al finalizar, esta función devuelve un '1' lógico y pone las banderas *flag_sw1* y *flag_sw6* a '0', para que no sigan sumando los contadores del Timer0.

Función SW6_pulsado():



Esta función lo único que hace es verificar si se ha pulsado el botón, se hace así para que desde el main sólo se haga referencia a la función y no se modifique la variable. Dicha verificación se hace mediante la variable *flag_sw6_correcto*, la cual se pone a '1' cuando la señal ha detectado que se ha pulsado el botón SW6 y ha pasado un antirrebotes. Si se verifica que se ha pulsado el botón, se pone de nuevo la variable a '0' y se devuelve un '1' lógico.

```
void subeM1() {  
    control_motor(1,1,1,0);  
}  
  
void bajaM1() {  
    control_motor(1,1,0,0);  
}  
  
void subeM5() {  
    control_motor(5,1,1,0);  
}  
  
void bajaM5() {  
    control_motor(5,1,0,0);  
}
```

Funciones subeM1(), bajaM1(), subeM5() y bajaM5():

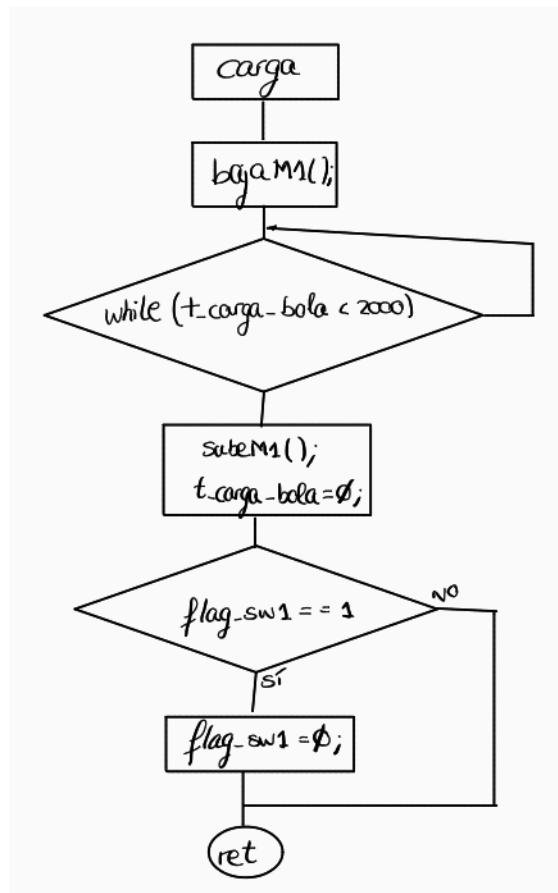
Estas 4 funciones controlan la dirección y el funcionamiento de los motores 1 y 5.

En las 4 funciones se llama a la función *control_motor* y se pasa como parámetros el motor al que se hace referencia, si lo apagamos lo encendemos y la dirección. El último parámetro en nuestro caso siempre es '0', ya que sólo hace referencia a una

funcionalidad del motor 2.

Los dos motores están implementados con PWM, lo cuál se puede ver en la función *control_motor*, la cual se encuentra en un archivo de la parte 1, llamado *funciones_lanzador*.

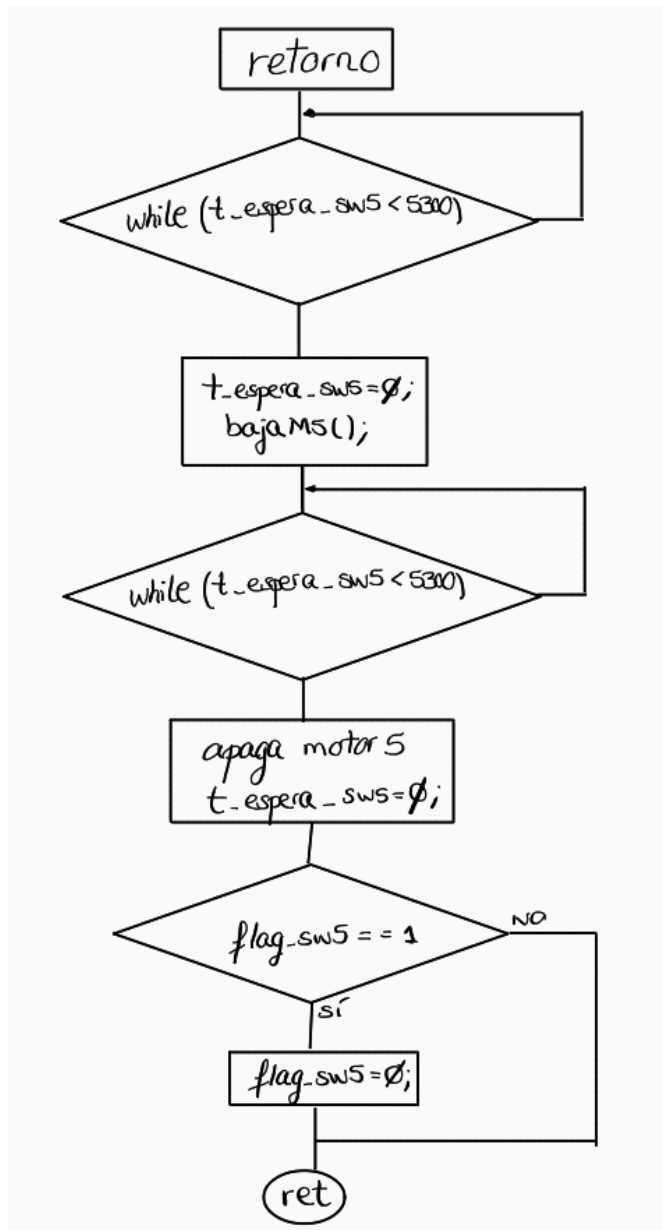
Función carga():



Esta función se encarga de realizar todos los pasos necesarios para cargar la bola al sistema lanzador. Es decir, primero bajará el motor M1 para poder recoger la bola. Una vez abajo, esperará un cierto tiempo para asegurarse de que la ha recogido y pasado este tiempo, subirá la bola a la posición de carga.

Tras este proceso pondrá la variable `flag_sw1` a '0' para que el contador `t_carga_bola` deje de contar y así no interfiera en acciones posteriores.

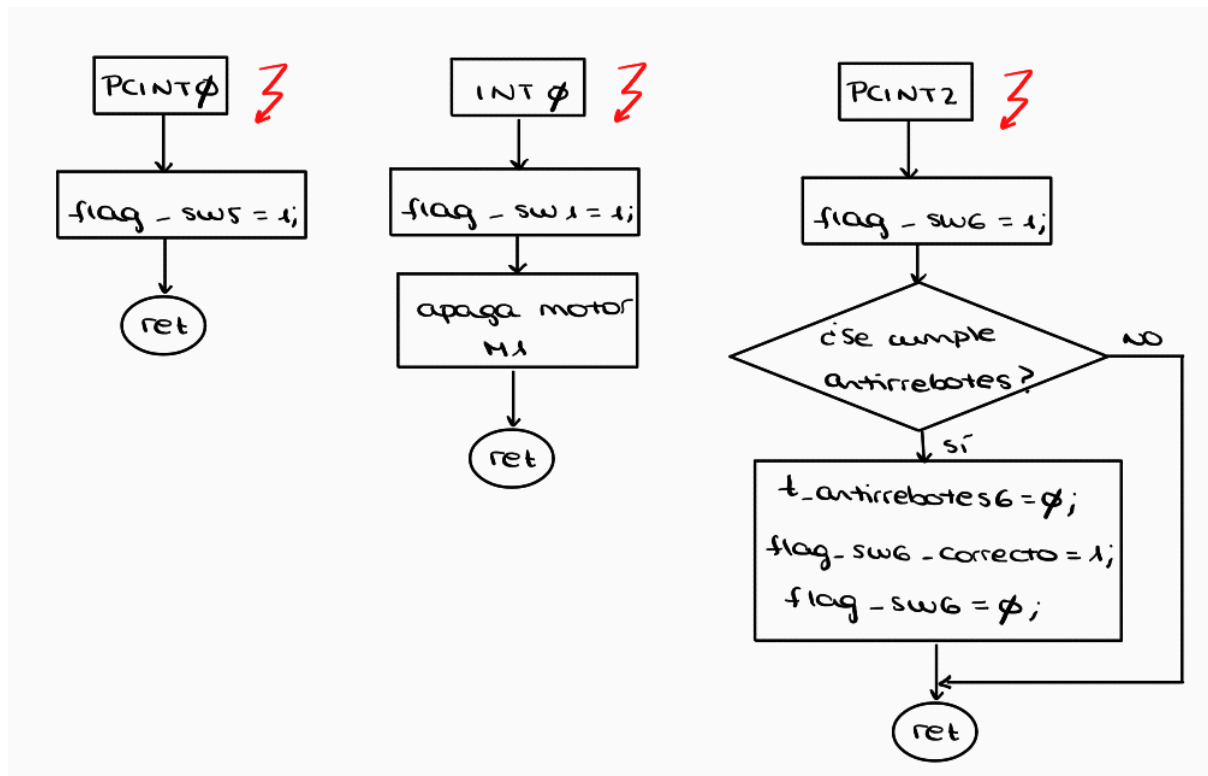
Función retorno():



Esta función realiza los pasos del retorno de la bola a la posición inicial. Inicialmente sube el motor 5 para poder coger la bola tras tirar los bolos.

Se inicia un temporizador y pasado este tiempo, baja de nuevo, retornando así la bola a la situación de carga. El segundo temporizador lo realizamos para poder apagar el motor 5 en cuanto llegue abajo, para que no haya sobretensión.

Subrutinas de interrupción:



La primera interrupción del motor 5, únicamente pone `flag_sw5` a '1'.

La segunda interrupción del motor 1, pone `flag_sw1` a '1' y apaga el motor M1.

En el caso de la interrupción 5, no podemos apagar el motor en cuanto entre en la interrupción como en la interrupción 1, porque al ser una PCINT, entra tanto de flanco de subida como de bajada. Por ello, se pararía nada más dejar de pulsar el microinterruptor.

En cuanto a la interrupción SW6, pone `flag_sw6` a '1', y en ese momento, en el timer, se pondrá a sumar `tantirrebotes6`. En el momento en el que `tantirrebotes6` sea mayor a 20 y comprobemos que `flag_sw6` sigue a '1', ponemos `flag_sw6_correcto` a '1', indicando que efectivamente se ha leído un pulso en el SW6.

Parte 3: Control de los bolos y de los displays de 7 segmentos e integración del sistema:

Integrantes: Camino Luque y Pablo Lecoq

En esta parte del código se controlan los sensores de los bolos (S01-S06) y el display de 7 segmentos (Da-Df y Dselector).

Función setup_timer():

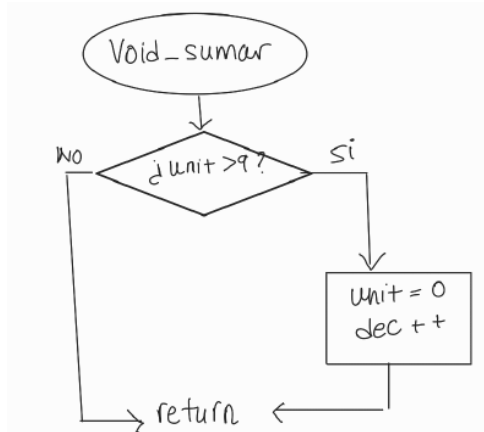
Se trata del timer0: es un timer de 8 bits, hemos elegido un preescalado de 1024, funciona en modo CTC y cuenta hasta 10 ms.

Función setup_bolos():

En el puerto K (PK2-PK7) tenemos configuradas las interrupciones PCINT de los sensores de los bolos.

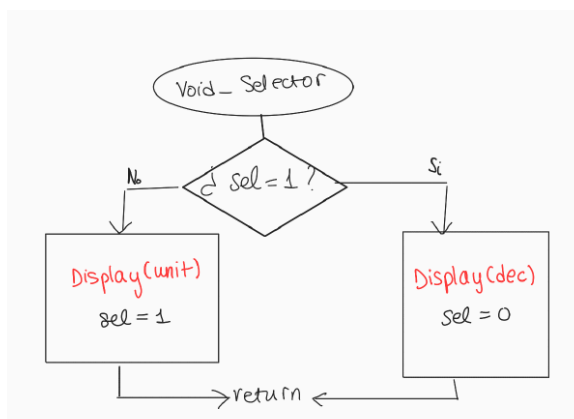
Los 7 segmentos de los displays se encuentran separados entre 2 puertos, el PB (mitad de los segmentos del display Da, Db, Dc, Dd) y PD (la otra mitad De, Df, Dg, y el selector Ds). Se inicializa a '0' las variables: unidades (unit), decenas (dec) y selector (sel), que se utilizarán para el display. Se inicializan también a cero las banderas de S1-S6, que se usarán en el recuento de bolos.

Función sumar():



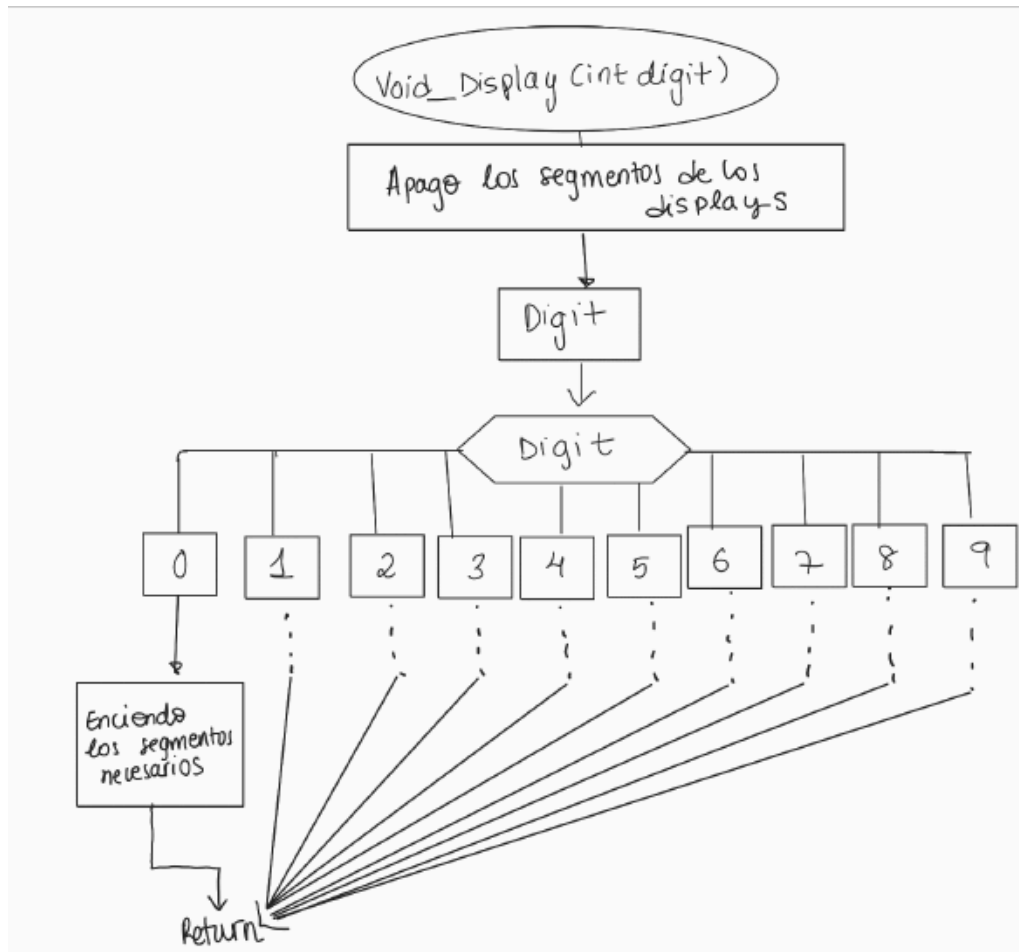
La función sumar comprueba si el valor de las unidades es mayor a 9 y, en ese caso, aumenta el valor de las decenas.

Función selector():



La función selector alterna el encendido y apagado de los displays izquierdo y derecho. A la función Selector se le llama desde la interrupción del timer0 cada 10 milisegundos para que actualice el valor que sale por los displays. El valor de unidades o decenas se envía a la función Display.

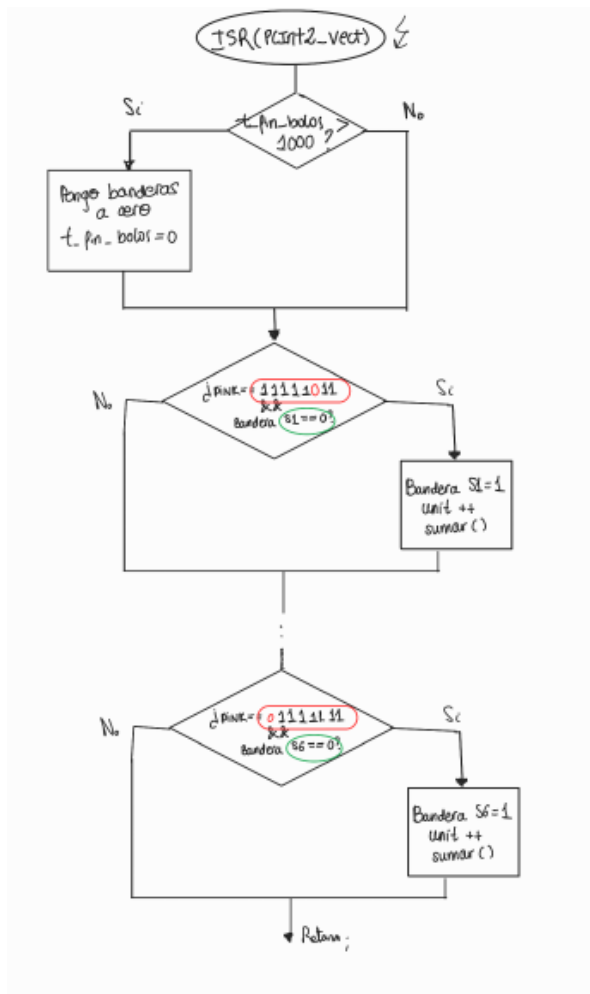
Función Display(int digit):



En esta parte del código se encuentra la lógica para poner por los displays cada uno de los números del 0 al 9. En primer lugar apagamos cada uno de los segmentos de los displays, luego realizamos un switch case en el que cada uno de los casos es un número distinto.

Hemos realizado la lógica teniendo en cuenta que tenemos separados los segmentos en dos puertos distintos en el PB (mitad de los segmentos Da, Db, Dc, Dd) y PD (la otra mitad De, Df, Dg).

Interrupción ISR (PCINT2_vect):



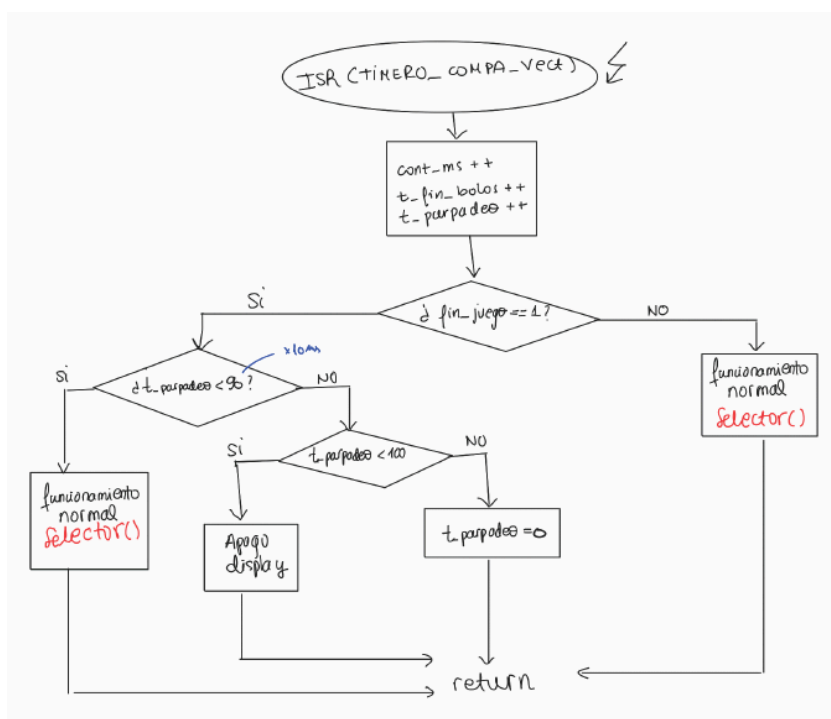
Esta ISR se produce cada vez que se genera una interrupción PCINT en los sensores de los bolos.

Comprobamos en primer lugar el valor del temporizador `t_fin_bolos`, si pasan 10 segundos desde el inicio se reinician las banderas y se puede volver a contar bolos.

En segundo lugar, comprobamos el valor del puerto K, en PINK. Cada una de las PCINTS al producirse varían el valor del PINK, por ejemplo: será 11111011 si se produce una interrupción en PK2, 11110111 si interrupción en PK3 etc.

La primera vez que se produce la interrupción las banderas S1-S6 se encuentran a '0', si se producen rebotes en los bolos y se vuelve a producir la interrupción el valor de la bandera estará a '1' por lo que no se contarán mas bolos. Para que unit aumente se debe de dar el caso en el que la bandera esté a '0' y el valor del PINK sea uno concreto, según el caso.

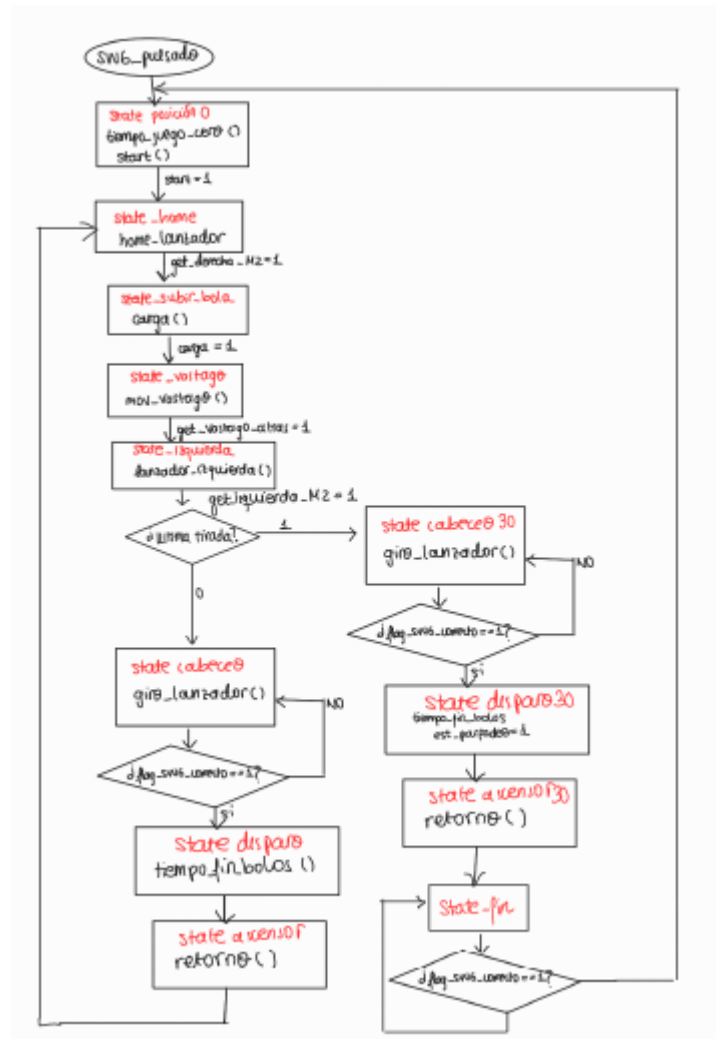
Interrupción ISR(TIMER0_COMPA_vect):



Esta interrupción se produce cada 10ms, comprueba en primer lugar si el valor de la variable `fin_juego` vale '1'. Se pondrá a '1' cuando pasen 30 segundos y empezará el modo parpadeo en los displays, durante 90 milisegundos estarán encendidos y durante los otros 10 milisegundos apagados.

Si no han pasado los 30 segundos funcionará normal.

FLUJOGRAMA IMPLEMENTACIÓN CONJUNTA:



En este flujograma podemos ver como nuestra maqueta es bastante secuencial y cómo se van sucediendo unas funciones a otras.

Existe una ejecución normal durante los 30 primeros segundos de partida y una ejecución un poco distinta cuando se cumplen los 30 segundos ya que se debe hacer parpadear un led y hacer parpadear los displays con la puntuación.

2. TABLA DE CONEXIONES

